

```
In [ ]: import pandas as pd
        from sklearn.model_selection import train_test_split
        from sklearn.preprocessing import StandardScaler
        from sklearn.ensemble import RandomForestClassifier
        from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
        import shap

        df = pd.read_csv('/content/heart_attack_prediction_dataset.csv')
```

```
In [ ]: df
```

```
Out[ ]:
```

	Patient ID	Age	Sex	Cholesterol	Blood Pressure	Heart Rate	Diabetes	Family History	Smokin
0	BMW7812	67	Male	208	158/88	72	0	0	
1	CZE1114	21	Male	389	165/93	98	1	1	
2	BNI9906	21	Female	324	174/99	72	1	0	
3	JLN3497	84	Male	383	163/100	73	1	1	
4	GFO8847	66	Male	318	91/88	93	1	1	
...	...	...	...	...	...	...	...	...	
8758	MSV9918	60	Male	121	94/76	61	1	1	
8759	QSV6764	28	Female	120	157/102	73	1	0	
8760	XKA5925	47	Male	250	161/75	105	0	1	
8761	EPE6801	36	Male	178	119/67	60	1	0	
8762	ZWN9666	25	Female	356	138/67	75	1	1	

8763 rows × 26 columns



```
In [ ]: df['Sex'] = df['Sex'].map({'Male': 1, 'Female': 0})
        df['Diet'] = df['Diet'].map({'Healthy': 2, 'Average': 1, 'Unhealthy': 0})

        bp_split = df['Blood Pressure'].str.split('/', expand=True)
        df['BP_Systolic'] = bp_split[0].astype(float)
        df['BP_Diastolic'] = bp_split[1].astype(float)
```

```
df.drop(['Blood Pressure'], axis=1, inplace=True)

df.drop(columns=['Patient ID', 'Country', 'Continent', 'Hemisphere'], inplace=True)

X = df.drop('Heart Attack Risk', axis=1)
y = df['Heart Attack Risk']
```

```
In [ ]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3,
```

```
In [ ]: clf = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
```

```
Out[ ]: ▼ Random Forest Classifier ⓘ ⓘ

RandomForestClassifier(class_weight='balanced', random_state=42)
```

```
In [ ]: y_pred = clf.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Accuracy: 0.6394066184861164

Confusion Matrix:

```
[[1667  24]
 [ 924  14]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.64	0.99	0.78	1691
1	0.37	0.01	0.03	938
accuracy			0.64	2629
macro avg	0.51	0.50	0.40	2629
weighted avg	0.55	0.64	0.51	2629

```
In [ ]: # SHAP explanation
explainer = shap.TreeExplainer(clf)
shap_values = explainer.shap_values(X_test)
shap.summary_plot(shap_values, X_test, feature_names=X.columns)
```

